

mixer-pro

Console de mixage temps réel embarquée

Guide utilisateur & guide développeur

COMPOSANT	mixer-pro · mixer-gui-http
VERSION	v7.0-e7.7b (GUI) · v7.0-e7.5a (daemon)
PLATEFORME CIBLE	Debix Model AB – NXP i.MX 8M Plus
LANGAGE DAEMON	C99 · ALSA-lib · pthread · libwebsockets-free
LANGAGE GUI	C99 (libmicrohttpd) · Alpine.js · Tailwind CDN
AUDIO	26 in · 18 out · 4 bus FX stéréo · 48 kHz · 32 bit · 2 ms periods
LATENCE ONE-WAY	≈ 14 ms (8 ms ALSA + 6 ms ring SPSC + DMA)
DATE DU DOCUMENT	2026-05-13
AUTEUR	Michael Jouannigot · Electrosens

Ce document couvre l'intégralité du composant `mixer-pro` et de son interface web `mixer-gui-http` : utilisation depuis le navigateur (sound engineer / opérateur board), puis architecture interne (threads RT, ring SPSC,

matrice de mix, moteurs d'effets, analyser FFT, protocole JSON et endpoints REST) pour le développeur appelé à maintenir ou étendre le système.

Résumé exécutif

mixer-pro est une console de mixage logicielle temps-réel écrite en C, conçue pour la plateforme audio embarquée Debix Model AB. Elle remplace fonctionnellement une table de mixage analogique 26→18 + 4 bus d'effets, expose une matrice de routage complète à 612 points de gain individuels, fournit 4 moteurs d'effets natifs (compresseur, reverb, delay, EQ 3 bandes), et publie un analyseur 4 canaux FFT 1024 + oscilloscope X-Y. L'ensemble est piloté en HTTP/JSON depuis un navigateur standard sur le LAN.

Cette documentation comprend deux volets :

- **Volet utilisateur** — tour de l'interface web, vocabulaire (strips, sends, returns, bus FX, taps, biquads), procédures opérationnelles (tac-reset, snapshot, monitoring).
- **Volet développeur** — architecture des threads RT, ring buffer SPSC, matrice de mix, vtable des effets, codec FFT radix-2, protocole JSON sur socket UNIX, endpoints REST, modèle d'état Alpine.js, procédures de build Yocto et déploiement.

REPÈRES CHIFFRÉS

MÉTRIQUE	VALEUR	NOTES
Inputs gérés	26 (8 mics + 8 stems USB + 2 phone + 8 returns FX)	Tous mono à l'entrée mixer, stéréo via link
Outputs gérés	18 (8 DSP play + 8 USB out + 2 phone)	Idem, link symétrique
Bus FX stéréo	4 (= 8 canaux)	Compressor, Reverb, Delay, EQ par défaut
Points de gain matriciels	612	26 × 8 sends + 34 × 18 master
Frequency / format	48 kHz / float32 interne / S32_LE ALSA	Conversion fixe ↔ flottant aux frontières
Period / buffer	96 frames (2 ms) / 4 periods = 8 ms	Plus 8 periods en ring SPSC = 16 ms tolérance jitter
Latence one-way	≈ 14 ms	Plancher Little & jitter, validé loopback
Lignes de code daemon	≈ 1 600 (mixer-pro.c+h) + 200 (analyzer) + 500 (effects)	C99 pur, pas d'allocations dans le hot path
Lignes de code GUI back	≈ 800 (mixer-gui-http.c)	libmicrohttpd, 1 thread reverse-proxy
Lignes de code GUI front	≈ 3 200 (index.html)	Alpine.js inline + Tailwind CDN

Table des matières

1. Vue d'ensemble & topologie audio

- 1.1 Place de mixer-pro dans la pile Debix
- 1.2 Vocabulaire : strips, sends, returns, bus FX, taps
- 1.3 Diagramme de flux audio

2. Guide utilisateur

- 2.1 Premier accès — URL, status, version GUI
- 2.2 Layout 2 colonnes : strips + BUS FX
- 2.3 Sidebar détail d'une voie (input / return / output)
- 2.4 Sends & returns FX
- 2.5 Onglet EFFETS — kcontrols TAC + DRC + biquads + crossover
- 2.6 Modaux BUS FX — Compressor / Reverb / Delay / EQ
- 2.7 Analyzer 4 taps — FFT + scope X-Y
- 2.8 Bouton tac-reset

3. Guide développeur — daemon mixer-pro

- 3.1 Threads RT : audio / play / control / analyzer
- 3.2 Ring buffer SPSC inter-thread
- 3.3 Mix matrix : send_gain & master_gain
- 3.4 Smoothing & ramp anti-zip
- 3.5 FX engines — vtable, signatures, paramètres
- 3.6 Analyzer FFT radix-2 inline + Hann window
- 3.7 ALSA setup : capture/playback, recover
- 3.8 Sécurité, RT prio, ulimit, signaux

4. Protocole JSON sur /run/mixer-pro.sock

- 4.1 Format ligne, parser hand-rolled
- 4.2 Opérations exhaustives (16 ops)
- 4.3 Exemples

5. Guide développeur — mixer-gui-http

- 5.1 Architecture libmicrohttpd
- 5.2 Endpoints REST exhaustifs
- 5.3 SSE streaming meters / analyzer
- 5.4 Modèle Alpine.js — état réactif côté front
- 5.5 Écriture des blobs DSP (TLV SOF binary)

6. Build & déploiement

- 6.1 Recipes Yocto
- 6.2 systemd units
- 6.3 Push incremental sur la board

7. Debug & profiling

- 7.1 Logs journal + dmesg
- 7.2 ALSA xrun & ring drops
- 7.3 mixerctl CLI
- 7.4 Pistes de profilage (perf, eBPF)

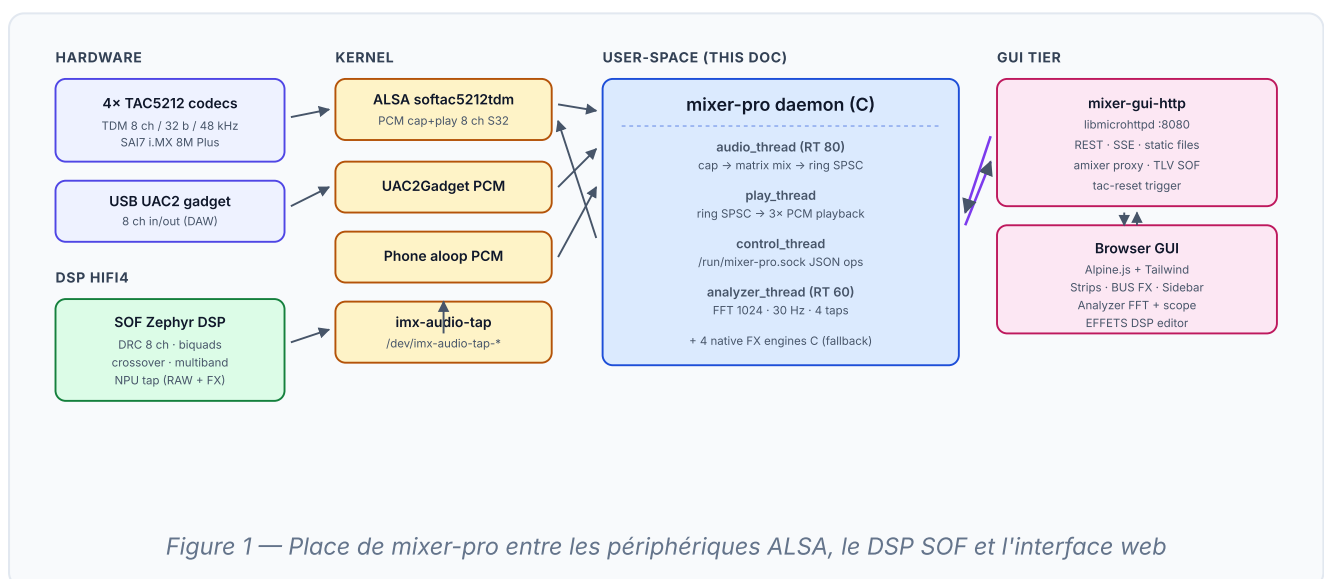
8. Roadmap & extensions envisagées

1. Vue d'ensemble & topologie audio

1.1 Place de mixer-pro dans la pile Debix

`mixer-pro` est le composant userspace temps-réel qui implémente la fonction "table de mixage" entre les périphériques audio exposés par le kernel Linux et le DSP SOF (HiFi4) chargé d'appliquer les effets. Il ne traite *pas* lui-même les effets temps-réel lourds (reverb, delay, EQ multi-bandes) — ceux-ci sont délégués au DSP via la chaîne ALSA `softac5212tdm`. mixer-pro gère :

- L'**acquisition** de 3 PCM capture en parallèle (DSP mics 8 ch, USB UAC2 gadget 8 ch, phone aloop 2 ch) ;
- Le **mixage temps-réel** entre 26 sources et 18 destinations via une matrice 34×18 de gains, avec ramps anti-zip ;
- L'application en **fallback CPU** de 4 bus d'effets "natifs" (compressor/reverb/delay/EQ) pour les sends FX, en attendant que le DSP les prenne en charge ;
- La **restitution** sur 3 PCM playback (DSP play 8 ch, USB out 8 ch, phone aloop out 2 ch) ;
- L'**analyseur FFT 4 taps** pour l'affichage live dans la GUI.



1.2 Vocabulaire

STRIP	Une voie d'entrée ou de sortie dans la GUI (analogie console : tranche). 26 strips d'entrée + 18 strips de sortie + 8 strips returns FX.
SEND	Niveau d'envoi d'une voie vers un bus FX. Une voie a 8 sends (4 bus × L/R). Gain de -∞ à +6 dB.
RETURN FX	Le signal de sortie d'un bus FX, ré-injecté dans la matrice master au même niveau qu'une voie d'entrée (strips 18..25).
BUS FX	Un canal stéréo qui reçoit la somme des sends, applique un effet (compressor / reverb / delay / EQ), et publie un return FX en sortie.

MASTER	La matrice 34×18 qui mixe finalement toutes les sources (26 inputs + 8 returns FX = 34 colonnes virtuelles ; mais seuls 26 sources réelles + 8 returns gérés directement) vers les 18 sorties.
TAP (ANALYZER)	Un point d'écoute non-destructif ($\neq$ NPU tap) où l'analyser FFT prélève le signal pour l'afficher dans la GUI. 4 taps configurables.
TAP (NPU)	Au-delà du périmètre mixer-pro : le tap matériel kernel <code>imx-audio-tap</code> qui expose le flux post-DSP au NPU pour l'inférence ML. À ne pas confondre.
TAC-RESET	Script kernel/userspace qui ré-initialise les 4 codecs TAC5212 et leurs sub-blocs. Obligatoire après chaque boot avant le premier <code>mixer-pro</code> .

2. Guide utilisateur

2.1 Premier accès

Connecter la board sur le LAN (DHCP). Récupérer l'IP via la console UART ou votre routeur. Depuis n'importe quel navigateur récent (Chrome 120+, Firefox 120+, Safari 17+) :

```
http://<IP-board>:8080/
```

La page principale affiche en haut un bandeau de statut :

- **ONLINE** si `mixer-pro` répond — l'UI passe en mode actif ;
- **OFFLINE** si le daemon ne répond pas dans 1 s — le bouton *Restart* tente `systemctl restart mixer-pro` ;
- La version GUI (ex. `v7.0-e7.7b`) et la version daemon (ex. `v7.0-e7.5a`) sont affichées à droite, utiles pour vérifier qu'aucune mise à jour n'a été oubliée.

À SAVOIR Si le daemon démarre pour la première fois après un boot board, il exécute automatiquement `tac-reset analog` avant d'ouvrir les PCM (`ExecStartPre=` dans le `systemd` unit). Sans ce reset, les TAC5212 boot en mode hi-Z et la capture renvoie 0 silencieusement.

2.2 Layout 2 colonnes

Depuis la version E7.4d (refactor layout), l'écran est divisé verticalement en deux zones :

- **Colonne gauche** (≈ 75% largeur) : les strips d'entrée au-dessus (M1..M8 mics, U1..U8 USB, P1..P2 phone, R1..R8 returns FX) et les strips de sortie en-dessous (S1..S8 speakers DSP, U1..U8 USB out, P1..P2 phone out). Chaque strip a son meter, son fader de gain, son bouton mute, et un indicateur de stéréo-link sur les paires adjacentes.
- **Colonne droite** (≈ 25% largeur) : les 4 cartes BUS FX verticales empilées, chacune avec son type d'effet, son niveau de return, et la liste des sorties vers lesquelles le return est routé (depuis E7.7a, code couleur vert/turquoise/orange selon L/R/both).

2.3 Sidebar détail d'une voie

Cliquer sur un strip (label ou zone vide) ouvre la **sidebar détail** sur la droite. Sa structure :

1. **Header** : nom de la voie (ex. "M3 — DSP mic 3"), type (input/return/output), bouton de stéréo-link avec la voie adjacente (M3+M4, U1+U2, etc.).
2. **Section ROUTING** :
 - Pour un input : 8 mini-faders L/R par bus FX (= 4 sends stéréo) + 18 mini-faders master vers les 18 sorties.
 - Pour un return FX : 18 mini-faders master uniquement.
 - Pour un output : section informationnelle (le routing s'édite côté input).

3. **Section EFFETS** (voir 2.5) : liste des kcontrols ALSA pertinents pour cette voie + éditeur DRC/biquads/crossover si DSP play.

2.4 Sends & returns FX

Pour envoyer une voie d'entrée vers un bus d'effets, ouvrir sa sidebar et ajuster les mini-faders *SEND* correspondants. Exemple : pour envoyer M1 vers un reverb sur BUS FX 2 avec 50 % en stéréo, mettre les deux faders "Send FX2 L" et "Send FX2 R" à -6 dB.

Le signal traité revient comme *return FX* (strips R1..R8). Cliquer sur R1 ouvre sa sidebar (corrigé E7.7b) et permet de doser sa contribution finale vers chaque sortie (S1..S8, U1..U8, P1..P2). Depuis E7.7a, chaque carte BUS FX montre directement les destinations actives du return en tags colorés.

2.5 Onglet EFFETS

La section EFFETS de la sidebar affiche les *kcontrols ALSA* pertinents pour la voie courante. Ces kcontrols pilotent les sous-modules du TAC5212 (chaîne ADC / DAC) et du DSP SOF (DRC, biquads, crossover, multiband).

TYPE KCONTROL	UI	DOMAINE
INTEGER (1 valeur)	Slider linéaire avec min/max	Gain ADC, AGC threshold, etc.
BOOLEAN	Toggle on/off	HPF enable, DC blocker, mute
ENUMERATED	Dropdown	Mode AGC, Bias source
BYTES (blob)	Éditeur dédié selon nom du kcontrol	Biquads RBJ, DRC, multiband, crossover

ÉDITEUR DRC PER-CHANNEL

Cliquer "Edit DRC" ouvre l'éditeur DRC *inline* (depuis E7.4b) pour la voie courante uniquement (pas un modal global). 8 paramètres exposés : threshold (dB), knee (dB), ratio, attack (s), release (s), pre-gain (dB), post-gain (dB), enable. L'éditeur encode en blob SOF Q-format (Q1.31 / Q8.24 / Q2.30 selon le champ) et le pousse via TLV SOF binary.

ÉDITEUR MULTIBAND DRC

Cliquer "Edit Multiband" expose 3 *bandes* × 7 *params* + 2 crossover-Hz éditables (depuis E7.4c, où ils étaient initialement câblés fixes à 250/3500). Le widget réutilise la grille de l'éditeur DRC simple.

ÉDITEUR BIQUADS RBJ

Pour les kcontrols "TACx biquad N" (24 par TAC × 4 = 96 biquads), un mini-éditeur expose type (LPF/HPF/BPF/Peak/LowShelf/HighShelf), F0 (Hz), Q, gain (dB). Coefficients calculés en JS via la cookbook RBJ et poussés en blob.

ÉDITEUR CROSSOVER

2 sliders Hz (lo/hi) entre 20 et 20 000 Hz, encoding LR-style 2nd-order.

2.6 Modaux BUS FX

Cliquer "Edit effect ►" sur une carte BUS FX ouvre un modal centré listant les paramètres du moteur d'effet associé :

EFFET	PARAMÈTRES	PLAGES
Compressor	threshold, ratio, attack, release, makeup	-60..0 dB · 1..20:1 · 0.1..200 ms · 10..2000 ms · 0..24 dB
Reverb	size, damp, wet	0..1 · 0..1 · 0..1
Delay	time, feedback, wet	0..1000 ms · 0..0.95 · 0..1
EQ 3-band	freq L/M/H, gain L/M/H, Q	20..20k Hz · ±18 dB · 0.3..10

Tous les sliders ont un debounce 50 ms et envoient un `op:set_fx_param` au daemon ; l'audio change immédiatement (smoothing inclus côté FX engine). Le bouton *Reset* renvoie un `op:reset_fx`.

2.7 Analyzer 4 taps

Depuis E7.5, quatre cartes **Analyzer** sont disposées en ligne sous les strips outputs. Chacune :

- Permet de choisir une source : mono (input 0..25, output 0..17) ou stéréo (paire link).
- Affiche un **FFT spectrum** 128 bins (20 Hz à 24 kHz, échelle log) avec hold-peak vert ;
- Affiche un **oscilloscope X-Y** 64 échantillons stéréo pour vérifier la phase / largeur stéréo ;
- Affiche le **RMS dBFS** en numérique sous le scope.

L'analyzer est rafraîchi à 30 Hz par le daemon (thread RT prio 60, inférieure à audio prio 80, pour ne jamais préempter le traitement).

2.8 Bouton tac-reset

En bas à droite de la barre supérieure, un bouton "Reset TAC" exécute `/usr/bin/tac-reset analog` via l'endpoint `POST /api/tac/reset`. Utile si la capture devient muette (typiquement après un reload du module sans cycle d'alim) ou si un TAC a perdu sa configuration.

Implémentation : `pthread_mutex_trylock` + `fcntl flock` côté serveur pour éviter deux runs concurrents qui se marcheraient dessus au reset i2c.

3. Guide développeur — daemon mixer-pro

3.1 Threads

Le daemon démarre 4 threads explicitement :

THREAD	PRIO	RÔLE	FONCTION D'ENTRÉE
<code>audio_thread</code>	SCHED_FIFO 80	Capture 3 PCM (DSP mics + UAC2 + phone), mix matrix vers 18 channels, écriture analyzer taps, push dans ring SPSC	<code>audio_thread()</code> ligne 320
<code>play_thread</code>	SCHED_FIFO 80	Pop du ring SPSC, écriture vers 3 PCM playback	<code>play_thread()</code> ligne 626
<code>control_thread</code>	SCHED_OTHER	Accept + select sur socket UNIX, dispatch des ops JSON	<code>control_thread()</code> ligne 1043
<code>analyzer_thread</code>	SCHED_FIFO 60	FFT 1024 + Hann + scope pour les 4 taps, every 33 ms	<code>analyzer_thread()</code> dans analyzer.c

Tous sont lancés en `main()` après `mlockall(MCL_CURRENT|MCL_FUTURE)` pour éviter les page-faults dans le hot path. La prio audio supérieure à analyzer garantit que la FFT ne préempte jamais le mix.

3.2 Ring buffer SPSC

Le ring SPSC est l'unique canal de communication audio entre `audio_thread` (producteur) et `play_thread` (consommateur). Choix de design :

- **Pourquoi un ring** : découpler la capture (qui doit lire 96 frames à l'heure ALSA exactement) de la playback (qui peut subir des recover SOF jusqu'à 60 ms). Sans ring, le moindre stutter de `snd_pcm_writeti` bloquerait la capture et provoquerait un xrun cascade.
- **Taille** : $8 \text{ periods} \times 96 \text{ frames} \times 18 \text{ ch} \times 4 \text{ octets} = \mathbf{55 \text{ KB}}$, tient en L1+L2 du Cortex-A53.
- **Tampon temporel** : $8 \text{ periods} \times 2 \text{ ms} = \mathbf{16 \text{ ms}}$ de jitter absorbé sans drop. Au-delà, l'écriture overrun → drop visible dans `ring_drops`, audio coupé jusqu'au recover (acceptable, rare).
- **Synchronisation** : indices `rd` / `wr` en `_Atomic unsigned`, ordonnés par `memory_order_acquire` côté reader, `memory_order_release` côté writer. Pas de mutex.

```
/* mixer-pro.c – extraits ring */
static float      g_ring[N_RING_PERIODS][PERIOD_FRAMES][N_OUTPUT_TOTAL];
static _Atomic unsigned g_ring_wr = 0;
static _Atomic unsigned g_ring_rd = 0;
static _Atomic unsigned g_ring_drops = 0;
```

3.3 Mix matrix

Le cœur du mixeur est `mix_frame()` (ligne 273), appelé en boucle pour chaque frame du period :

```
static void mix_frame(const float in[N_INPUT_REAL],
                    float out[N_OUTPUT_TOTAL],
                    float bus_pre[N_BUS_FX_CH],
                    float ret[N_RETURN_CH])
{
    /* 1. Apply input gains + mute → in_post[] */
    /* 2. Sends : for each (i in 0..25, b in 0..7) {
        bus_pre[b] += in_post[i] * g_st.send_gain[i][b]; } */
    /* 3. Apply FX engines : ret[] = fx[k]→process(bus_pre[2k..2k+1]) */
    /* 4. Master : for each (s in 0..33, o in 0..17) {
        out[o] += sources[s] * g_st.master_gain[s][o]; } */
}
```

La matrice `g_st.master_gain[34][18] = 612 floats` est intégralement lue chaque frame. À 48 kHz, c'est 30 millions de multiplications/sec, parfaitement dans les capacités du A53 (un seul cœur suffit, le reste du SoC est libre).

3.4 Smoothing & ramp anti-zip

Chaque changement de gain (send ou master) déclenche une rampe linéaire de `GAIN_RAMP_FRAMES = 64` frames (= 1.33 ms à 48 kHz) entre ancienne et nouvelle valeur, pour éviter les "zip" audibles.

Implémentation dans `smooth_gains()` (ligne 243), appelée par `audio_thread` en début de chaque period.

3.5 FX engines

Les 4 moteurs d'effets natifs C (fallback CPU avant que le DSP les absorbe) sont définis dans `effects.c` via une vtable simple :

```
typedef struct fx_engine_s fx_engine_t;
struct fx_engine_s {
    fx_type_t type;
    void *state; /* opaque par engine */
    void (*process)(fx_engine_t *e, const float in[2], float out[2], int frames);
    int (*set_param)(fx_engine_t *e, const char *name, float v);
    void (*reset)(fx_engine_t *e);
    void (*destroy)(fx_engine_t *e);
};
```

Engines fournis : `fx_compressor_new()`, `fx_reverb_new()`, `fx_delay_new()`, `fx_eq3_new()`.
Chacun alloue son state à l'init (jamais dans process), expose son set de paramètres via une table `fx_param_spec_t[]` consultable par `op:get_fx`.

3.6 Analyzer FFT

L'analyzer est implémenté dans `analyzer.c` (≈ 200 lignes, zéro dépendance externe). Spécifications :

FFT	Radix-2 Cooley-Tukey iterative DIT, 1024 points (inline ≈ 70 lignes)
Window	Hann, précomputée à l'init dans <code>w_hann[1024]</code>
Output	Magnitude dB, downsamplee 4:1 en peak-hold → 128 bins int8
Scope	64 paires stéréo, conversion float → S16 par <code>l × 32767</code>
Rate	30 Hz (33 ms <code>nanosleep</code>)
Concurrence	Double-buffer ring : <code>side</code> 0/1, atomic, audio_thread écrit, analyzer_thread lit

Le hot path côté audio est `analyzer_tap_write(t, l, r)` inline dans le header : 5 loads atomic + 2 stores ring + éventuel side-swap. Coût mesuré ≈ 3 ns par frame × 4 taps actifs = 12 ns/frame, négligeable devant les 30 ns du mix.

3.7 ALSA setup

Ouverture PCM via `pcm_open()` (ligne 167). Paramètres fixes :

```
access      = SND_PCM_ACCESS_RW_INTERLEAVED
format      = SND_PCM_FORMAT_S32_LE
rate        = 48000
channels    = 8 (DSP/UAC2) ou 2 (phone)
period      = 96 frames
buffer      = 4 × 96 = 384 frames
start_thr   = 1 buffer (start au premier write)
stop_thr    = ULONG_MAX (jamais d'auto-stop)
```

Sur xrun, `pcm_recover()` (ligne 222) appelle `snd_pcm_recover(.., 1)` avec silent=1 pour ne pas spammer le journal en cas de cascade.

3.8 Sécurité runtime

- `mlockall(MCL_CURRENT|MCL_FUTURE)` à l'init pour pin la mémoire ;
- `pthread_setschedparam(SCHED_FIFO, prio)` par thread ;
- `signalfd` ou `sigaction` sur SIGINT/SIGTERM pour un shutdown propre avec drain ring + close PCM ;
- `LimitMEMLOCK=infinity` et `LimitRTPRIO=99` dans le systemd unit (`mixer-pro.service`) ;
- Pas d'allocation, pas de printf, pas de syscall bloquante dans `audio_thread` ni `play_thread` (mlog est bloqué à init).

4. Protocole JSON sur /run/mixer-pro.sock

4.1 Format

Socket UNIX `SOCK_STREAM` à `/run/mixer-pro.sock`, permissions 0666. Le protocole est **line-based** : une requête = une ligne JSON terminée par `\n`, une réponse = une ou plusieurs lignes JSON.

Le parser est hand-rolled (pas de cJSON) — il s'agit d'extraire 4 types de champs :

- `json_get_str(buf, key, out, max)` — string entre guillemets ;
- `json_get_int(buf, key, *out)` — entier décimal ;
- `json_get_float(buf, key, *out)` — float décimal ;
- `json_has_op(buf, "name")` — match l'opération.

C'est suffisant car le format des messages est figé et contrôlé par mixer-gui-http. Aucune validation stricte n'est faite — un message malformé est silencieusement ignoré.

4.2 Opérations

OP set_send

```
{ "op": "set_send", "in": <0..25>, "bus": <0..7>, "gain": <float> }
```

Met à jour `send_gain[in][bus]`. Active la ramp anti-zip.

OP set_master

```
{ "op": "set_master", "src": <0..25>, "out": <0..17>, "gain": <float> }
```

Met à jour `master_gain[src][out]`. src 0..17 = inputs réels, 18..25 = returns FX. Active la ramp anti-zip.

OP set_fx_bus

```
{ "op": "set_fx_bus", "bus": <0..3>, "type": "compressor|reverb|delay|eq3" }
```

Recrée le moteur d'effet sur ce bus. Détruit l'ancien.

OP set_input_gain

```
{ "op": "set_input_gain", "src": <0..25>, "gain": <float> }
```

Strip gain unitaire (E7.2). 0..2.0 linéaire.

OP set_mute

```
{ "op": "set_mute", "src": <0..25>, "mute": <0|1> }
```

Mute hard sur la voie (gain forcé à 0 avec ramp).

OP get_strip_routing

```
{ "op": "get_strip_routing", "src": <0..25> }
```

Réponse : { "sends": [8 floats], "master": [18 floats], "gain": float, "mute": int }

OP get_state

```
{ "op": "get_state" }
```

Snapshot complet : version, send_gain[26][8], master_gain[34][18], input_gain[26], mute[26], fx_types[4], fx_params[4][...].

OP set_fx_param

```
{ "op": "set_fx_param", "bus": <0..3>, "name": "threshold|size|...", "value": <float> }
```

Met à jour un paramètre du moteur FX du bus. Routing vers la vtable `set_param`.

OP get_fx

```
{ "op": "get_fx", "bus": <0..3> }
```

Réponse : type courant + dictionnaire des paramètres.

OP get_meters

```
{ "op": "get_meters" }
```

Réponse compacte JSON ≤ 16 KB : tableau de niveaux RMS par strip (in / out / return) + payload analyzer 4 taps (spec128 + scope128 + rms_dB).

OP set_tap

```
{ "op": "set_tap", "id": <0..3>, "kind": <0|1|2|3>, "a": <idx>, "b": <idx|-1> }
```

Reconfigure un slot analyzer. kind : 0=NONE, 1=INPUT, 2=BUS_PRE, 3=OUTPUT. b=-1 $\rightarrow$ mono.

OP get_taps

```
{ "op": "get_taps" }
```

Réponse : configuration courante des 4 slots analyzer.

OP reset_fx

```
{ "op": "reset_fx", "bus": <0..3> }
```

Restaure les paramètres FX par défaut sur ce bus.

OP reset

```
{ "op": "reset" }
```

Remet toute la matrice à 0 et tous les strip gains à 1.

4.3 Exemples (depuis le shell board)

```
# Ouvrir M3 → S5 à -6 dB
echo '{"op":"set_master","src":2,"out":4,"gain":0.5}' | nc -U /run/mixer-pro.sock

# Snapshot complet
echo '{"op":"get_state"}' | nc -U /run/mixer-pro.sock

# Configurer tap 0 sur la sortie S1+S2 stéréo
echo '{"op":"set_tap","id":0,"kind":3,"a":0,"b":1}' | nc -U /run/mixer-pro.sock
```

5. Guide développeur — mixer-gui-http

5.1 Architecture

`mixer-gui-http` est un serveur HTTP minimal écrit en C basé sur **libmicrohttpd**. Il joue trois rôles :

1. **Reverse-proxy JSON** vers `/run/mixer-pro.sock` : reçoit les requêtes du browser, sérialise l'op JSON, lit la réponse, la renvoie.
2. **Static file server** pour `index.html` et assets (Tailwind/Alpine CDN charges depuis le navigateur, donc seul le HTML est servi).
3. **Pont kernel** pour les actions privilégiées : `amixer cset/contents`, `tac-reset`, écriture blobs TLV SOF.

5.2 Endpoints REST

METHOD	PATH	DESCRIPTION
GET	/	Renvoie <code>www/index.html</code> (Alpine.js single-page)
GET	/api/version	Version GUI + version daemon
GET	/api/state	Proxy → <code>op:get_state</code>
GET	/api/meters	Proxy → <code>op:get_meters</code> (JSON court)
GET	/api/strip/routing? src=N	Proxy → <code>op:get_strip_routing</code>
POST	/api/send/set	Proxy → <code>op:set_send</code>
POST	/api/master/set	Proxy → <code>op:set_master</code>
POST	/api/strip/gain/set	Proxy → <code>op:set_input_gain</code>
POST	/api/strip/mute/set	Proxy → <code>op:set_mute</code>
POST	/api/fx/bus/set	Proxy → <code>op:set_fx_bus</code>
POST	/api/fx/param/set	Proxy → <code>op:set_fx_param</code>
POST	/api/fx/reset	Proxy → <code>op:reset_fx</code>
GET	/api/tap/list	Proxy → <code>op:get_taps</code>
POST	/api/tap/set	Proxy → <code>op:set_tap</code>
GET	/api/alsa/contents	Exec <code>amixer -c softac5212tdm contents</code> , parse, renvoie en JSON
POST	/api/alsa/set	Exec <code>amixer -c softac5212tdm cset numid=N value</code>
POST	/api/dsp/blob/set	Écrit un blob binaire dans un kcontrol TLV (numid + payload hex)
POST	/api/tac/reset	Lance <code>/usr/bin/tac-reset analog</code> en sous-processus mutex+flock
POST	/api/system/restart	<code>systemctl restart mixer-pro</code>
GET	/api/stream/meters	SSE streaming 30 Hz : meters + analyzer

5.3 SSE streaming

Pour éviter le coût d'un long-poll HTTP, le streaming meters/analyzer est servi par **Server-Sent Events** :

```
Content-Type: text/event-stream
Cache-Control: no-cache

data: {"meters":[...], "analyzer":[{"..."}, {"..."}, {"..."}, {"...}]}
```

```
data: {"meters":[...],"analyzer":[...]}
```

Côté serveur, un thread dédié par connexion (libmicrohttpd `MHD_USE_THREAD_PER_CONNECTION`) appelle `op:get_meters` via le socket UNIX à 30 Hz et flush la réponse SSE.

5.4 Modèle Alpine.js

Le front est un seul fichier `index.html` (≈ 3 200 lignes, inlined CSS Tailwind + JS Alpine.js). Le state Alpine est :

```
function app() { return {
  /* — connexion — */
  online: false, lastTickMs: 0, version: '',
  /* — état mixer — */
  state: { send_gain: null, master_gain: null,
    input_gain: null, mute: null,
    fx_types: ['compressor','reverb','delay','eq3'],
    fx_params: [{}, {}, {}, {}] },
  /* — meters & analyzer — */
  meters: { in:[], ret:[], out:[] },
  analyzer: [ { spec:[], scope:[], rms:-120 }, ... ],
  /* — sélection — */
  selectedStrip: -1, selectedKind: 'input',
  /* — routing courant de la sidebar — */
  stripRouting: { sends:[], master:[], gain:1, mute:0 },
  /* — returns destinations cache (E7.7) — */
  returnsRouting: [ { L:[18], R:[18] } × 4 ],
  /* — blobs DSP (kcontrols BYTES) — */
  dspBlobs: { /* numid → { name, type, bytes } */ },
  ...
}}
```

Les fetches utilisent `fetch()` avec `AbortController` pour ne pas paralléliser les `get_strip_routing` en cas de double-click. La réactivité est naturelle : tous les changements d'état dans `state` ou `analyzer` repropagent automatiquement vers le DOM via le proxy Alpine.

PIÈGE Toute affectation directe d'objet `dspBlobs[id] = { ... }` casse le proxy Alpine et la réactivité. Mutation correcte : `Object.assign(dspBlobs[id], { ... })`. Bug rencontré en E7.4b avec le multiband DRC (les sliders n'apparaissaient pas).

5.5 Écriture blobs DSP (TLV SOF binary)

Les kcontrols BYTES côté SOF sont écrits via `amixer cset` avec un payload TLV. Bug historique (corrigé E7.4d) : le tag TLV initialement codé `0x1004` (SOF_CTRL_TLV_DATA, format IPC4) est rejeté par le kernel SOF en IPC3 qui vérifie `header.numid == scontrol->cmd`. Le bon tag pour blobs IPC3 est `SOF_CTRL_CMD_BINARY = 3`.

```
/* mixer-gui-http.c — sof_blob_write */
struct {
```

```
uint32_t numid;      /* MUST = 3 (SOF_CTRL_CMD_BINARY) for IPC3 */
uint32_t length;     /* size of payload in bytes */
uint8_t data[];      /* the blob */
} tlv = { 3, len, ... };
ioctl(snd_ctl_elem_write, &tlv);
```

Le parser du POST body utilise 3 `strchr` successifs pour sauter au-delà de la colonne, du `"` d'ouverture, puis du `"` de la valeur — bug historique (E7.4d) où seuls 2 `strchr` étaient utilisés, ce qui faisait échouer l'écriture avec "malformed hex".

6. Build & déploiement

6.1 Recipes Yocto

Trois recettes vivent dans `meta-local/recipes-audio/` :

RECIPE	SOURCES	OUTPUT
<code>mixer-pro_1.0.bb</code>	<code>mixer-pro.{c,h}</code> , <code>analyzer.{c,h}</code> , <code>effects.{c,h}</code> , <code>mixerctl.c</code> , <code>Makefile</code> , <code>mixer-pro.service</code>	<code>/usr/bin/mixer-pro</code> , <code>/usr/bin/mixerctl</code> , <code>/lib/systemd/system/mixer-pro.service</code>
<code>mixer-gui-http_1.0.bb</code>	<code>mixer-gui-http.c</code> , <code>www/index.html</code> , <code>Makefile</code> , <code>mixer-gui-http.service</code>	<code>/usr/bin/mixer-gui-http</code> , <code>/var/www/mixer-gui/index.html</code> , <code>/lib/systemd/system/mixer-gui-http.service</code>
<code>tac5212-service_1.0.bb</code>	<code>tac-reset</code> script + service	<code>/usr/bin/tac-reset</code> , <code>/lib/systemd/system/tac5212.service</code>

Tous trois sont ajoutés à `imx-image-full` via `IMAGE_INSTALL:append = " mixer-pro mixer-gui-http tac5212-service"` dans `recipes-core/images/imx-image-full.bbappend`.

6.2 systemd units

L'unité `mixer-pro.service` :

```
[Unit]
Description=mixer-pro DAW mixing daemon
After=tac5212.service sound.target
Requires=tac5212.service

[Service]
ExecStartPre=/usr/bin/tac-reset analog
ExecStart=/usr/bin/mixer-pro --no-uac2 --no-phone
TimeoutStartSec=30
LimitMEMLOCK=infinity
LimitRTPRIO=99
Restart=on-failure
RestartSec=2

[Install]
WantedBy=multi-user.target
```

Les flags `--no-uac2` et `--no-phone` sont temporaires en attendant que les recettes UAC2 et phone aloop soient stables ; ils permettent à mixer-pro de démarrer même si ces PCM ne sont pas disponibles.

6.3 Push incremental (sans full re-flash)

```
# Rebuild ciblé
bitbake mixer-pro mixer-gui-http

# scp les binaires
IMG_PRO=Model_AB_Infinity/tmp/work/armv8a-poky-linux/mixer-pro/1.0/image
IMG_GUI=Model_AB_Infinity/tmp/work/armv8a-poky-linux/mixer-gui-http/1.0/image

scp $IMG_PRO/usr/bin/mixer-pro    root@192.168.0.9:/usr/bin/
scp $IMG_GUI/usr/bin/mixer-gui-http root@192.168.0.9:/usr/bin/
scp $IMG_GUI/var/www/mixer-gui/index.html root@192.168.0.9:/var/www/mixer-gui/

# Redémarrer
ssh root@192.168.0.9 \
    "systemctl restart mixer-pro mixer-gui-http"
```

Le redémarrage est non destructif côté audio : le ring SPSC se draine, les PCM se ferment proprement, et mixer-pro restart en < 1 s avec ré-application des kcontrols par défaut. Les paramètres ne sont *pas* persistants (dette E7.8 — voir roadmap).

7. Debug & profiling

7.1 Logs

```
# Logs daemon mixer-pro (mlog → stderr via systemd)
journalctl -u mixer-pro -f

# Logs GUI HTTP server
journalctl -u mixer-gui-http -f

# Kernel ALSA / SOF
dmesg -w | grep -iE 'sof|tac5212|sai|softac5212tdm'
```

Points de vigilance :

- `ALSA xrun` dans dmesg → période trop courte ou load CPU trop fort. Vérifier `top -H` pour les threads RT.
- `ring_drops` en log mixer-pro → tampon SPSC débordé, typiquement après un recover SOF > 16 ms.
- `SOF IPC error` dans dmesg → firmware DSP désynchronisé, lancer `tac-reset analog`.

7.2 mixerctl CLI

Le binaire `mixerctl` (90 lignes) permet de scripter toutes les opérations JSON depuis le shell :

```
mixerctl get_state
mixerctl set_master src=2 out=4 gain=0.5
mixerctl set_tap id=0 kind=3 a=0 b=1
mixerctl set_fx_param bus=0 name=threshold value=-12.0
```

NOTE mixerctl n'est *pas encore* déployé sur la board (oubli dans `FILES:${PN} =` du recipe, dette E7.7). Workaround : `nc -U /run/mixer-pro.sock` + JSON brut.

7.3 Pistes de profilage

- `perf top -C 0,1 -g` pour identifier le hot path du mix sur les A53 ;
- `perf stat -e cache-misses,cycles` pour mesurer l'efficacité L1/L2 du ring SPSC ;
- `cyclictst -p 80 -t 1 -i 2000 -l 100000` pour mesurer le jitter SCHED_FIFO du kernel (PREEMPT activé, attendre < 100 µs) ;
- eBPF tracing : `bpftrace -e 'kprobe:snd_pcm_recover { @[comm] = count(); }'` pour compter les recover ALSA par process.

8. Roadmap & extensions envisagées

ITEM	SÉVÉRITÉ	DESCRIPTION
E7.8 — persistance state	moyenne	Snapshot binaire au shutdown + restore au boot. Cible : gains, sends, mutes, types FX, params FX, taps, kcontrols ALSA, blobs DSP.
E7.x — presets nommés	moyenne	Live / Studio / Conf — pouvoir basculer entre 3 snapshots typés.
E8 — programme GUI temps réel avancé	haute	Couplage tap NPU + analyser GUI : FFT NPU + FFT voie + édition simultanée DSP/TAC en direct (objectif final V5.4.x).
Migration FX vers DSP	moyenne	Les 4 moteurs natifs C deviennent un fallback ; le DSP HiFi4 absorbe les bus FX au full.
Dette : mixerctl pas dans imx-image-full	faible	FILES:\${PN} += à compléter dans le recipe.
Dette : kernel-module-imx-audio-tap deb packaging	haute	Bloque <code>bitbake imx-image-full -c rootfs</code> . Pré-existant à E7.7.
WS au lieu de SSE	faible	SSE est unidirectionnel — un WebSocket permettrait de réduire la latence des ack côté UI sur set_param.
Authentification	haute (prod)	L'API REST est ouverte sur le LAN sans auth. Pas critique en dev, à régler avant déploiement client.